



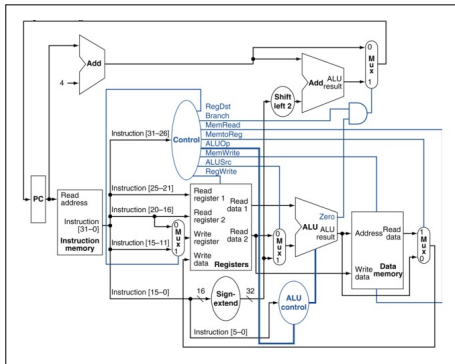
# Bubble Sort in Single Cycle MIPS Processor

Xezel Peshlakai, Pradip Sharma Poudel, Sajid Kamal

Department of Computer Science, Department of Engineering, Swarthmore College, Swarthmore PA

## Introduction

This project explores the implementation of a single-cycle MIPS processor using Verilog and simulating in Quartus. The processor supports R-type, memory, branching, and custom instructions. The Bubble Sort Algorithm is implemented within the single-cycle processor to demonstrate its computational capability. The architecture is based on a datapath incorporating key components such as the Program Counter, Instruction Memory, Control Unit, Register File, ALU, ALU Control, and Data Memory.



## Background

- Instruction set architecture
- Single cycle MIPS processor executes each instruction within a single clock cycle
- The datapath contains the functional units and connections to allow an instruction set architecture in CPU

## Methodology

### Processor Overview

Supported Instructions:

- R-type: add, sub, and, or, slt
- Memory: lw, sw
- Branching: beq

Key Modules:

- Processor: combines all modules
- Program Counter (PC): manages instruction addresses
- Instruction Memory: fetches instructions
- Control: decodes instruction types and signals other components
- RegisterFile: stores and retrieves operand values
- muxALU: chooses to read data or give results
- ALU Control: executes arithmetic and logic operations
- Data Memory: handles memory operations

Additional Modules:

- Adder: computes branch address
- muxPC: determines branching decisions
- SignExtender: converts 16-bit values to 32-bit
- ThirtyTwoBitMux: control data flow between components

### Bubble Sort Implementation:

Added Instructions:

- Jump: j, jal, jr
- Additional: slt, addi
- Custom: swap

Custom Swap:

OP Code: [31:26] = 6'b111111

```
else if (RegWrite) begin
    reg_file[writeReg] <= writeData1;
end else if (Swap) begin
    reg_file[readReg1] <= writeData2;
    reg_file[readReg2] <= writeData1;
end
```

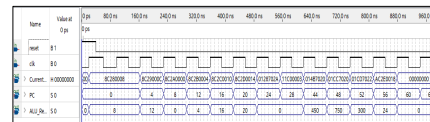
## Results

### Single Cycle Processor Waveform Simulation:

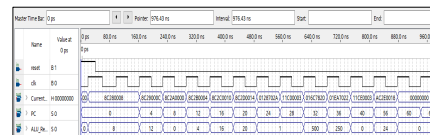
a = 250, b = 200, c = 300, and d = 450

```
if i > j f = b + c - a
else f = a + b + c - d
```

Case 1: i = 10, j = 10



Case 2: i = 20, j = 10



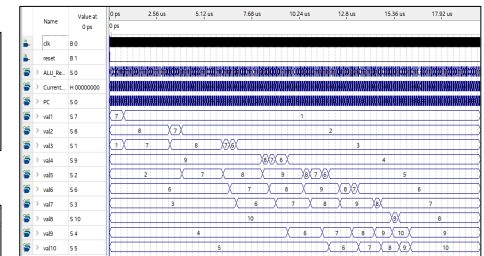
### Python vs. MIPS

```
int bubbleSort(array):
    for i in range(array.size()):
        for j in range(i+1, array.size()):
            if (array[j] < array[i]):
                temp = array[i]
                array[i] = array[j]
                array[j] = temp
        end
    end
end
```

```
1 .data
2 smallTable: .word 7,8,1,9,2,6,3,10,4,5
3 .text
4 la $a0, smallTable
5 addi $a1, $a0, 10
6 jal sort
7 j finalExit
8 sort:
9     addi $t0, $t0, 0 # i
10 outerLoop:
11     sll $t3, $t0, $a1 # i < size
12     beq $t3, $zero, exit # false exit
13     sll $t4, $t0, 2 # i * 4
14     addi $t1, $t0, 1 # j = i + 1
15 innerLoop:
16     sll $t3, $t1, $a1 # j < size
17     beq $t3, $zero, exit1 # false exit1
18     sll $t5, $t1, 2 # j * 4
19     lw $s1, ($t4) # s1 = array[i]
20     lw $s2, ($t5) # s2 = array[j]
21     sll $t6, $s2, $a1 # array[j] < array[i]
22     beq $t6, $zero, exit2 # false exit2
23     swap $s1, $s2 # swap registers s1 and s2
24     sw $s1, ($t4) # array[i] = s1
25     sw $s2, ($t5) # array[j] = s2
26 exit2:
27     addi $t1, $t1, 1 # j = j + 1
28     j innerLoop
29 exit1:
30     addi $t0, $t0, 1 # i = i + 1
31     j outerLoop
32 exit:
33     finalExit:
34
```

### Bubble Sort Waveform Simulation:

Numbers 1-10 are randomized in the fixed memory, and they are sorted in the simulation after 100 clock cycles



## Conclusion

This implementation showcases a single-cycle processor capable of executing fundamental arithmetic, memory operations, branch instructions, and sorting using the Bubble Sort algorithm.

Future improvements to the processor include adding support for multiplication and division. Additionally, exploring optimizations such as multi-cycle execution and pipelining could significantly enhance the processor's speed and performance.

## Acknowledgements

Thank you to Professor Maggie Delano, Associate Professor of Engineering for assisting us in the development of the processor.